# BÀI TẬP LÝ THUYẾT
# LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

| | |
|---|---|
| Họ và tên | Nguyễn Dương Bảo Trân |
| MSSV / Lớp | 3123410386 / DCT1236 |

## Mục tiêu học tập

- Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete.
- Xây dựng planning graph theo GraphPlan với các mức trạng thái $S_i$, mức hành động $A_i$ và các hành động Noop.
- Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs.
- Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.
- Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc.

**Hướng dẫn**: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

## Bài 1. Khái niệm cơ bản trong GraphPlan

Hãy định nghĩa ngắn gọn các khái niệm sau và nêu ý nghĩa của chúng trong việc xây dựng planning graph.

a) Interference (xung đột do can thiệp):

Hai hành động bị Interference khi hiệu ứng của một hành động làm mất hoặc phủ định tiền điều kiện của hành động kia.

Ví dụ: nếu hành động A1 xóa một literal mà A2 cần làm precondition, thì A1 và A2 không thể thực hiện đồng thời.

Ý nghĩa: giúp GraphPlan loại bỏ các cặp hành động không thể xảy ra cùng lúc trong cùng một action level.

b) Inconsistent Effects / Inconsistency (hiệu ứng không nhất quán):

Hai hành động có Inconsistent Effects khi hiệu ứng của chúng mâu thuẫn trực tiếp với nhau, tức một hành động thêm một literal còn hành động kia xóa hoặc thêm phủ định của literal đó.

Ví dụ: $A_1 \rightarrow P$, $A_2 \rightarrow \neg P$ thì A1 và A2 mutex.

Ý nghĩa: ngăn GraphPlan chọn đồng thời hai hành động tạo ra các trạng thái trái ngược nhau.

c) Competing Needs (tiền điều kiện cạnh tranh):

Hai hành động bị Competing Needs khi có ít nhất một cặp precondition của chúng đang mutex với nhau ở state level trước đó.

Nếu: $P \in Pre(A_1)$ và $Q \in Pre(A_2)$

Mà P và Q mutex thì: A1 và A2 cũng mutex.

Ý nghĩa: bảo đảm chỉ những hành động có tiền điều kiện có thể cùng tồn tại mới được xem xét thực hiện song song.

d) Có những cách nào để tìm một kế hoạch trong mô hình lập kế hoạch cổ điển?

Có thể tìm kế hoạch bằng các cách chính:

1. Forward search: bắt đầu từ trạng thái ban đầu, áp dụng các hành động hợp lệ cho đến khi đạt mục tiêu.
2. Backward search: bắt đầu từ mục tiêu, tìm các hành động có thể tạo ra mục tiêu rồi lần ngược về trạng thái ban đầu.
3. Planning graph / GraphPlan: xây dựng lần lượt các mức trạng thái và hành động, xác định mutex, sau đó trích xuất kế hoạch khi mục tiêu xuất hiện cùng một mức và không mutex.

e) Khi dùng heuristic dựa trên GraphPlan, những giả định nào của classical planning được nới lỏng? Vì sao điều này hữu ích hơn so với tìm kiếm ngây thơ?

Khi xây dựng heuristic từ planning graph, ta có thể nới lỏng một số ràng buộc của bài toán thật, ví dụ tạm bỏ qua một số delete effects hoặc xung đột giữa hành động để ước lượng khoảng cách đến mục tiêu.

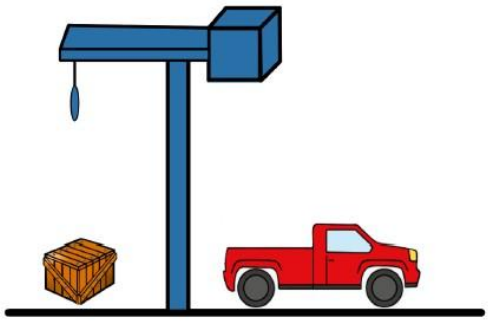
Nhờ đó, bài toán nới lỏng dễ giải hơn và cho ta một giá trị heuristic để hướng dẫn tìm kiếm.

Ý nghĩa: thay vì thử các trạng thái một cách gần như ngẫu nhiên, heuristic cho biết trạng thái nào có vẻ gần mục tiêu hơn, nhờ đó giảm đáng kể số trạng thái cần mở rộng.

## Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Cho một cần cẩu, một kiện hàng và một xe tải. Ban đầu kiện hàng ở bên trái, xe tải ở bên phải và cần cẩu quay về bên trái. Mục tiêu là đưa kiện hàng lên xe tải và cuối cùng cần cẩu quay về bên trái.

Cần cẩu có thể quay giữa trái và phải, có hoặc không mang kiện hàng. Cần cẩu có thể nhấc kiện hàng nếu cả hai ở cùng phía. Kiện hàng chỉ có thể được đưa lên xe tải bằng cần cẩu.



a) Xác định các biến/đối tượng, các predicate cần dùng, trạng thái ban đầu và trạng thái mục tiêu.

Đối tượng:

- Crane: cần cẩu
- Box: kiện hàng
- Truck: xe tải
- Left, Right: hai vị trí

Có thể dùng các predicate:

$$CraneAt(Left),\ CraneAt(Right)$$
$$BoxAt(Left),\ BoxAt(Right)$$
$$Holding(Box)$$
$$OnTruck(Box)$$

Trạng thái ban đầu:

$$S_0 = \{CraneAt(Left),\ BoxAt(Left),\ TruckAt(Right)\}$$

Mục tiêu:

$$G = \{OnTruck(Box),\ CraneAt(Left)\}$$

b) Xây dựng các toán tử hành động. Với mỗi toán tử, ghi rõ Preconditions, Add List và Delete List.

1. PickLeft

Nhấc kiện hàng khi cần cẩu và hàng cùng ở bên trái.

Preconditions:

$$CraneAt(Left),\ BoxAt(Left)$$

Add List:

$$Holding(Box)$$

Delete List:

$$BoxAt(Left)$$

2. MoveLeftToRight

Cần cẩu quay từ trái sang phải.

Preconditions:

$$CraneAt(Left)$$

Add List:

$$CraneAt(Right)$$

Delete List:

$$CraneAt(Left)$$

Nếu đang Holding(Box) thì kiện hàng được mang theo cần cẩu.

3. LoadTruck

Đặt kiện hàng lên xe tải.

Preconditions:

$$CraneAt(Right),\ Holding(Box),\ TruckAt(Right)$$

Add List:

$$OnTruck(Box)$$

Delete List:

$$Holding(Box)$$

4. MoveRightToLeft

Cần cẩu quay từ phải về trái.

Preconditions:

$$CraneAt(Right)$$

Add List:

$$CraneAt(Left)$$

Delete List:

$$CraneAt(Right)$$

Một kế hoạch đạt mục tiêu là:

$$PickLeft \rightarrow MoveLeftToRight \rightarrow LoadTruck \rightarrow MoveRightToLeft$$

c) Vẽ planning graph cho 3 bước đầu tiên. Có thể dùng hình minh họa thay cho proposition nếu biểu diễn rõ ràng.

Mức S0: $S_0 = \{CraneAt(Left), BoxAt(Left), TruckAt(Right)\}$

Mức A0:

Các hành động có thể thực hiện: $PickLeft$, $MoveLeftToRight$ và các No-op để duy trì proposition.

Mức S1:

Có thể xuất hiện: $Holding(Box)$, $CraneAt(Right)$ cùng với các proposition được giữ bởi No-op.

Mức A1:

Khi đã có Holding(Box) và có thể đưa cần cẩu sang phải, hành động cần thiết tiếp theo là: $MoveLeftToRight$

Các No-op tiếp tục duy trì: $Holding(Box),\ TruckAt(Right)$

Mức S2:

Ta có thể đạt: $CraneAt(Right)$, $Holding(Box)$, $TruckAt(Right)$

Do đó ở mức tiếp theo có thể thực hiện: $LoadTruck$

Sơ đồ: $S_0 \rightarrow A_0 \rightarrow S_1 \rightarrow A_1 \rightarrow S_2$

với đường kế hoạch chính: $PickLeft \rightarrow MoveLeftToRight \rightarrow LoadTruck$

d) Vẽ hai mức đầu tiên của GraphPlan và chỉ ra các cặp hành động mutex; với mỗi cặp, ghi loại mutex.

Ở mức A0:

- PickLeft và MoveLeftToRight là mutex do Interference, vì MoveLeftToRight xóa CraneAt(Left) trong khi PickLeft cần CraneAt(Left) làm precondition.

- MoveLeftToRight và No-op(CraneAt(Left)) là mutex do Inconsistent Effects, vì một hành động xóa CraneAt(Left) còn No-op giữ lại CraneAt(Left).

Ở mức A1:

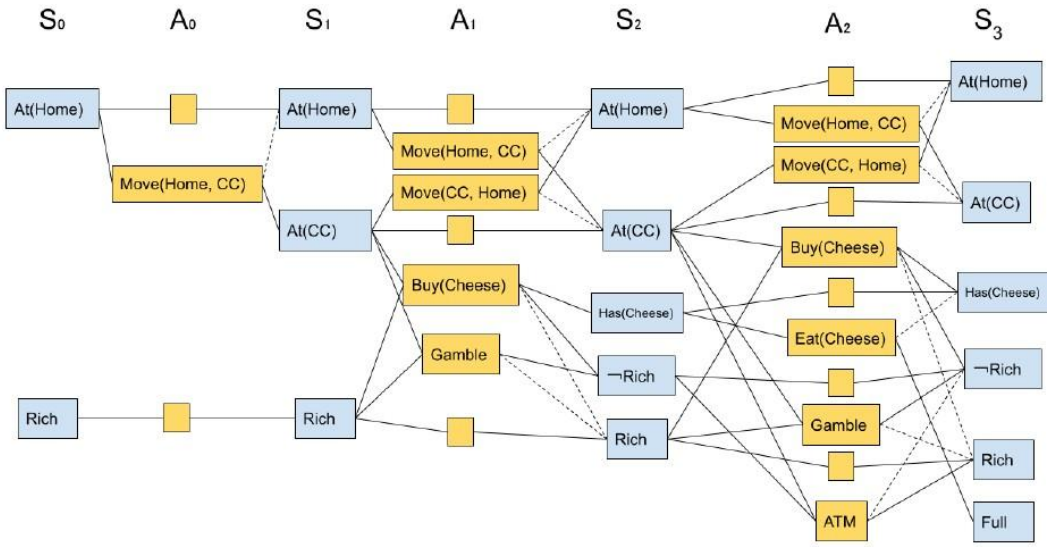
- Nếu một hành động cần CraneAt(Left) và hành động khác cần CraneAt(Right), mà hai proposition này mutex ở S1, thì hai hành động đó mutex do Competing Needs.

## Bài 3. Quan hệ Mutex trong planning graph

Pinky đi lấy thức ăn. Các hành động được cho như sau:

| **Hành động** | **Preconditions** | **Add List** | **Delete List** |
|---|---|---|---|
| Move(A,B) | At(A) | At(B) | At(A) |
| Buy(Cheese) | At(ChuckyCheese), Rich | Has(Cheese), ¬Rich | |
| Gamble | At(ChuckyCheese), Rich | ¬Rich | Rich |
| ATM | At(ChuckyCheese), ¬Rich | Rich | ¬Rich |
| Eat(Cheese) | Has(Cheese) | Full | Has(Cheese) |

Trạng thái bắt đầu: Rich ∧At(Home). Mục tiêu: Full. Planning graph tương ứng được cho trong hình dưới:




*Planning graph cho bài toán Pinky.*

a) Ở mức $A_0$, hãy chỉ ra một cặp hành động mutex do Inconsistent Effects.

Chọn cặp Move(Home, CC) và No-op(At(Home))

Move(Home,CC) xóa At(Home), trong khi No-op(At(Home)) duy trì At(Home).

Vì hai hành động tạo ra hai hiệu ứng đối nghịch nên: Mutex do Inconsistent Effects.

b) Ở mức $A_1$, hãy chỉ ra một cặp hành động mutex do Interference.

Chọn cặp Buy(Cheese) và Gamble. Cả hai đều cần: Rich nhưng Gamble làm mất Rich (\(Rich\) nằm trong Delete List), trong khi Buy(Cheese) cần Rich làm precondition.

Do đó Mutex do Interference.

c) Ở mức $A_2$, hãy chỉ ra một cặp hành động mutex do Competing Needs.

Chọn Eat(Cheese) và Move(Home, CC). Tiền điều kiện tương ứng là: Has(Cheese) và At(Home).

Ở S2, Has(Cheese) và At(Home) là mutex vì Has(Cheese) được tạo ra từ Buy(Cheese) tại ChuckyCheese, trong khi việc duy trì At(Home) không tương thích với tiền điều kiện cần để thực hiện Buy(Cheese).

Vì precondition của hai hành động mutex nên: Mutex do Competing Needs.

d) Giải thích ngắn gọn vì sao mỗi cặp ở trên thỏa loại mutex tương ứng.

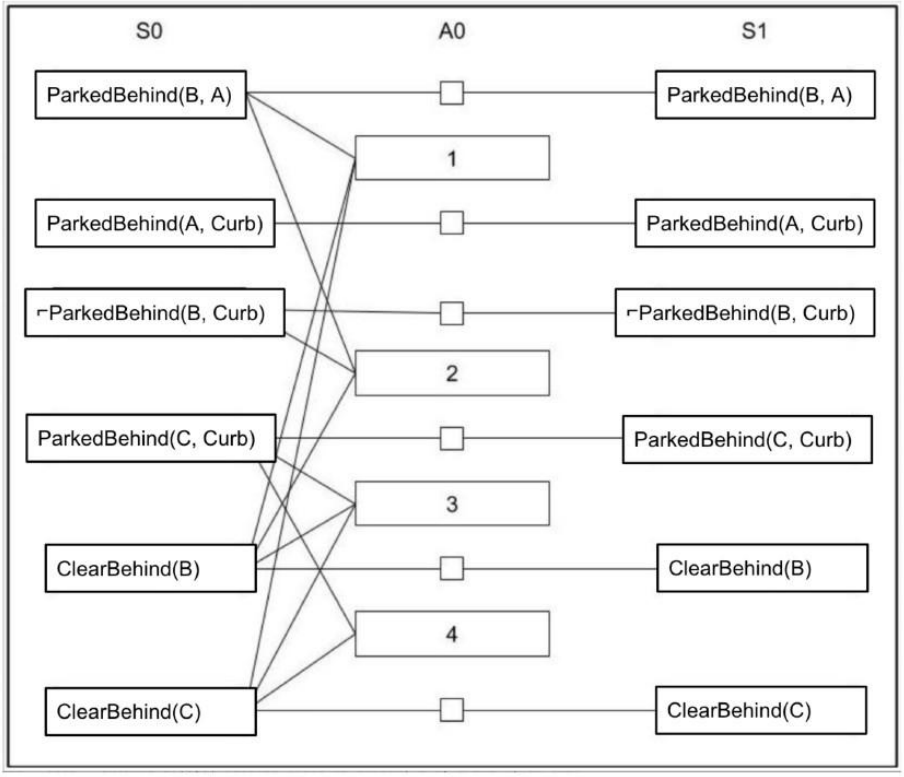
- Inconsistent Effects: một hành động tạo/giữ literal, hành động kia xóa literal đó.
- Interference: hiệu ứng của một hành động phá tiền điều kiện của hành động kia.
- Competing Needs: precondition của hai hành động đang mutex ở state level trước đó.

## Bài 4. Classical Planning và GraphPlan - Valet Parking

Xét bài toán Valet Parking với hai predicate ClearBehind(car) và ParkedBehind(car1, car2). Hai toán tử được định nghĩa như sau:

| **Thành phần** | **ParkBehind(car1, car2)** | **ParkInNewRow(car)** |
|---|---|---|
| Preconditions | ClearBehind(car1)<br>ClearBehind(car2)<br>ParkedBehind(car1, place) | ClearBehind(car)<br>ParkedBehind(car, place)<br>¬ParkedBehind(car, curb) |
| Add List | ParkedBehind(car1, car2)<br>ClearBehind(place)<br>¬ParkedBehind(car1, place)<br>¬ClearBehind(car2) | ParkedBehind(car, curb)<br>ClearBehind(place)<br>¬ParkedBehind(car, place) |
| Delete List | ClearBehind(car2)<br>ParkedBehind(car1, place) | ParkedBehind(car, place)<br>¬ParkedBehind(car, curb) |

Hình dưới là mẫu cho hai mức đầu tiên $S_0 \rightarrow A_0 \rightarrow S_1$. Các nút hành động No-op đã được biểu diễn bằng các ô vuông nhỏ không tên.




*Mẫu GraphPlan cho Valet Parking*

a) Điền tên hành động phù hợp vào các ô 1-4 ở mức $A_0$. Nếu không tồn tại hành động hợp lệ cho tập tiền điều kiện đó, ghi "N/A".

1. ParkInNewRow(B)
2. ParkBehind(B, C)
3. ParkBehind(C, B)

4. N/A

b) Với từng hành động 1-4, liệt kê TẤT CẢ predicate ở $S_1$ nhận add-edge từ hành động đó. Nếu hành động không hợp lệ, ghi "N/A".

Hành động 1 – ParkInNewRow(B)

$$ParkedBehind(B, Curb)$$
$$ClearBehind(A)$$
$$\neg ParkedBehind(B, A)$$

Hành động 2 – ParkBehind(B,C)

$$ParkedBehind(B, C)$$
$$ClearBehind(A)$$
$$\neg ParkedBehind(B, A)$$
$$\neg ClearBehind(C)$$

Hành động 3 – ParkBehind(C,B)

$$ParkedBehind(C, B)$$
$$ClearBehind(Curb)$$
$$\neg ParkedBehind(C, Curb)$$
$$\neg ClearBehind(B)$$

Hành động 4: N/A

c) Nêu hai action node có quan hệ mutex do Inconsistent Effects. No-op được phép sử dụng.

Chọn ParkBehind(B,C) và No-op(ClearBehind(C)).

Vì ParkBehind(B,C) tạo:

$$\neg ClearBehind(C)$$

trong khi No-op giữ:

$$ClearBehind(C)$$

nên hai hành động mutex do Inconsistent Effects.

d) Nêu hai action node có quan hệ mutex do Interference.

Chọn ParkBehind(B,C) và ParkBehind(C,B)

Vì ParkBehind(B,C) xóa:

$$ClearBehind(C)$$

trong khi ParkBehind(C,B) cần:

$$ClearBehind(C)$$

làm precondition.

Nên hai hành động mutex do Interference.

e) GraphPlan "leveled off" khi nào? Chọn một đáp án:

☐ A. Tất cả các hành động có thể đã được khám phá.

☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.

**☐ C. Hai mức liên tiếp của planning graph giống hệt nhau.**

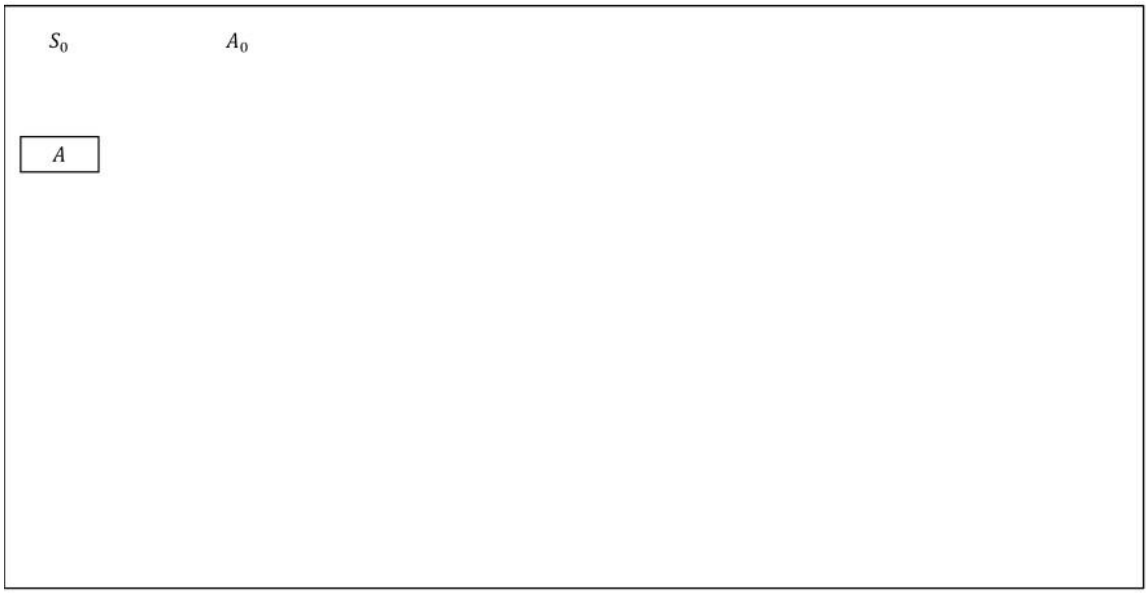
☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

## Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

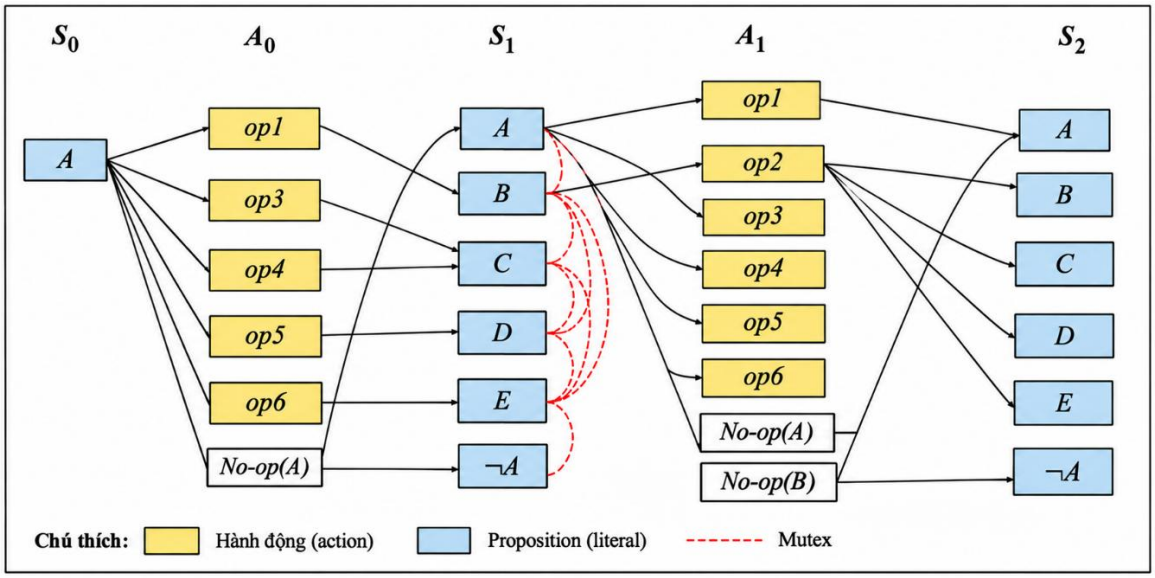
Cho môi trường lập kế hoạch với 6 toán tử. Trạng thái bắt đầu là A; mục tiêu là C D E. Mỗi thời điểm chỉ áp dụng một toán tử và cần tìm kế hoạch có ít thao tác nhất.

| | op1 | op2 | op3 | op4 | op5 | op6 |
|---|---|---|---|---|---|---|
| Precondition | A | B | A | A | A | A |
| Add | B | C, D, E | C | D | E | E, ¬A |
| Delete | | | | | | |

a) Chạy GraphPlan: vẽ lần lượt các mức trạng thái và mức hành động cho đến khi thuật toán dừng. Nhớ bổ sung các No-op cho trạng thái cần duy trì.




*Khung để hoàn thiện planning graph; trạng thái đầu $S_0 = \{A\}$.*




*Planning graph cho Bài 5(a); trạng thái ban đầu $S_0 = \{A\}$.*

b) Kế hoạch nào được GraphPlan trả về?

Để dùng op2 cần có B. Từ A, dùng:

$$op1: A \rightarrow B$$

Sau đó:

$$op2: B \rightarrow C, D, E$$

Vậy kế hoạch là op1 → op2.

Tổng cộng 2 thao tác.

c) Kế hoạch trên có tối ưu về số lượng thao tác hay không? ☐ **Có** ☐ Không

Có vì nếu không dùng op2, ta phải dùng riêng:

$$op3,\ op4,\ op5$$

để tạo C, D, E, tức cần ít nhất 3 thao tác.

Trong khi

$$op1 \rightarrow op2$$

Chỉ cần 2 thao tác.

Vậy kế hoạch tối ưu về số thao tác.

d) Liệt kê TẤT CẢ cặp toán tử mutex ở $A_0$ và TẤT CẢ cặp proposition mutex ở $S_1$. Nếu không có, ghi "Không có".

Do đề quy định mỗi thời điểm chỉ áp dụng một toán tử, các toán tử thực sự khác nhau ở A0 không thể chạy đồng thời.

Các cặp toán tử mutex gồm:

$$(op1, op3),\ (op1, op4),\ (op1, op5),\ (op1, op6)$$
$$(op3, op4),\ (op3, op5),\ (op3, op6)$$
$$(op4, op5),\ (op4, op6)$$
$$(op5, op6)$$

Ngoài ra op6 tạo $\neg A$, nên op6 mutex với $No\text{-}op(A)$ do Inconsistent Effects / Interference.

Ở S1, các proposition được tạo bởi những toán tử mutex cũng có thể mutex. Đặc biệt:

$$(B, C),\ (B, D),\ (B, E)$$
$$(C, D),\ (C, E),\ (D, E)$$

là mutex theo ràng buộc chỉ thực hiện một toán tử tại một thời điểm.

Ngoài ra: $A$ và $\neg A$ chắc chắn mutex.

Vậy các cặp proposition mutex chính:

$$(A, \neg A), (B, C), (B, D), (B, E), (C, D), (C, E), (D, E)$$

## Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp trong không gian trạng thái với GraphPlan. Trong trường hợp nào planning graph giúp giảm đáng kể không gian tìm kiếm?

Tìm kiếm trực tiếp trong không gian trạng thái xét từng trạng thái và từng hành động có thể áp dụng để tìm đường đến mục tiêu. Khi số trạng thái lớn, số nhánh có thể tăng rất nhanh.

GraphPlan xây dựng planning graph theo từng mức trạng thái \(S_i\) và hành động \(A_i\), đồng thời dùng quan hệ mutex để loại những hành động hoặc proposition không thể cùng tồn tại.

Planning graph giúp giảm đáng kể không gian tìm kiếm khi:

- có nhiều hành động có thể xảy ra,
- có nhiều mục tiêu con,
- tồn tại nhiều quan hệ mutex giúp loại sớm các tổ hợp không hợp lệ.

b) Vì sao No-op quan trọng trong GraphPlan? Nếu bỏ toàn bộ No-op thì việc biểu diễn trạng thái tồn tại qua nhiều mức sẽ bị ảnh hưởng như thế nào?

No-op dùng để duy trì một proposition từ mức trạng thái này sang mức trạng thái tiếp theo khi không có hành động nào làm thay đổi proposition đó.

Ví dụ:

$$P \xrightarrow{No\text{-}op(P)} P$$

Nếu bỏ toàn bộ No-op, các proposition đã đúng ở mức trước có thể không xuất hiện ở mức sau, dù thực tế không có hành động nào làm chúng mất đi.

Vì vậy No-op giúp GraphPlan biểu diễn đúng việc trạng thái được duy trì qua nhiều mức.

c) Cho một mục tiêu gồm nhiều literal. Hãy giải thích vai trò của mutex giữa các literal mục tiêu trong quyết định mở rộng planning graph hoặc kết luận thất bại.

Giả sử mục tiêu gồm nhiều literal:

$$G = \{g_1, g_2, \dots, g_n\}$$

Để có thể trích xuất kế hoạch tại một mức Si, cần:

1. Tất cả literal mục tiêu đều xuất hiện trong Si.
2. Không có cặp literal mục tiêu nào mutex với nhau.

Nếu các mục tiêu đã xuất hiện nhưng vẫn mutex, GraphPlan chưa thể trích xuất kế hoạch, nên phải tiếp tục mở rộng planning graph.

Nếu planning graph đã leveled off mà các mục tiêu vẫn mutex, có thể kết luận không tồn tại kế hoạch trong graph đó.

d) Nêu quy trình tổng quát để trích xuất một kế hoạch từ planning graph sau khi tất cả literal mục tiêu đã xuất hiện ở cùng một mức và không mutex với nhau.

Khi tất cả mục tiêu xuất hiện ở cùng một mức và không mutex:

1. Bắt đầu từ các literal mục tiêu ở mức trạng thái cuối.
2. Chọn các hành động ở mức trước có Add List tạo ra các mục tiêu đó.
3. Các hành động được chọn phải không mutex với nhau.
4. Lấy các precondition của các hành động đó làm mục tiêu con cho mức trước.
5. Tiếp tục lùi ngược qua các mức cho đến S0.
6. Các hành động đã chọn theo thứ tự từ S0 đến mục tiêu tạo thành kế hoạch.

Nếu ở một mức không chọn được tập hành động không mutex phù hợp, cần backtrack và thử tập hành động khác.

**— Hết —**